Django Complete Notes — Corrected Version

Language: English Technical Terms + বাংলা Explanation

Focus: Backend Development, Practical Learning & Interview Preparation

Table of Contents

- Part 1: Django Installation and Setup
- Part 2: Django Request Lifecycle
- Part 3: Reusable Django Apps and Python Packaging
- Final Revision Checklist
- Recommended Learning Roadmap

Part 1: Django Installation and Setup

1. What Is Django?

Django হলো একটি high-level Python web framework, যা দিয়ে secure, maintainable এবং database-driven web applications তৈরি করা যায়।

Django follows the **batteries-included philosophy**, meaning it provides many built-in features required for web development.

What Can We Build with Django?

- Blog and News Websites
- E-commerce Applications
- Authentication Systems
- REST APIs
- School Management Systems
- Task Management Applications
- SaaS Applications
- Content Management Systems

Key Features

- Built-in authentication system
- Object-Relational Mapper (ORM)
- URL routing
- Template engine
- Form handling and validation
- Security features
- Database migrations
- Admin panel
- Testing utilities

2. Django Prerequisites

Django শেখার আগে নিচের বিষয়গুলো জানা ভালো:

- Python fundamentals
- Functions and classes
- Object-Oriented Programming (OOP)
- Virtual environments
- `pip` package installer
- Basic HTML and HTTP concepts

- Basic SQL and database concepts
- Git and GitHub fundamentals

Important: Django শুরু করার জন্য আলাদা Apache, Nginx অথবা PostgreSQL server ইনস্টল করা বাধ্যতামূলক নয়। Django development server এবং SQLite দিয়েই শুরু করা যায়।

3. Python and Django Version Compatibility

Django-এর প্রতিটি release নির্দিষ্ট Python versions support করে। Installation-এর আগে official compatibility documentation যাচাই করবে।

Official resources:

- Python Downloads: https://www.python.org/downloads/
- Django Documentation: https://docs.djangoproject.com/en/
- Django Downloads: https://www.djangoproject.com/download/

Check Python version:

```
python --version
```

Windows alternative:

```
py --version
```

Check installed Django version:

```
python -m django --version
```

Stable Version vs Development Version

Stable Release: সাধারণ development এবং production projects-এর জন্য উপযুক্ত।

Development Version: Django framework-এর source code নিয়ে কাজ, নতুন feature পরীক্ষা অথবা Django-তে contribution করার জন্য ব্যবহৃত হয়।

Recommendation: Beginner হিসেবে একটি supported stable Django release ব্যবহার করবে।

4. What Is a Virtual Environment?

Virtual environment হলো একটি isolated Python environment, যেখানে একটি project-এর dependencies অন্য project থেকে আলাদা রাখা যায়।

Example:

```
workspace/
├── project_a/
│   └── .venv/
└── project_b/
    └── .venv/
```

প্রতিটি project আলাদা Django version বা dependency ব্যবহার করতে পারে।

Advantages

- Dependency conflicts কমায়।
- Project-specific packages আলাদা রাখে।
- Reproducible development environment তৈরি করতে সাহায্য করে।
- Team collaboration সহজ করে।

5. Create a Django Project

Step 1: Create a Project Directory

```
mkdir myproject
cd myproject
```

Step 2: Create a Virtual Environment

```
python -m venv .venv
```

Step 3: Activate the Virtual Environment

Windows CMD:

```
.venv\Scripts\activate
```

Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

Linux/macOS:

```
source .venv/bin/activate
```

Environment deactivate করতে:

```
deactivate
```

Step 4: Install Django

```
python -m pip install Django
```

Verify installation:

```
python -m django --version
```

Why Use `python -m pip`?

এটি নির্বাচিত Python interpreter-এর সঙ্গে সম্পর্কিত `pip` ব্যবহার করতে সাহায্য করে। ফলে একাধিক Python installation থাকলে package ভুল environment-এ install হওয়ার ঝুঁকি কমে।

6. Create the Django Project Structure

```
django-admin startproject config .
```

এখানে:

- `config` হলো project configuration package-এর নাম।
- `.` অর্থ বর্তমান directory-তে project structure তৈরি করা।

Expected structure:

```
myproject/
├── .venv/
├── config/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── manage.py
```

Important Files

File	Purpose
`manage.py`	Django management commands চালায়
`settings.py`	Project configuration
`urls.py`	URL routing configuration
`wsgi.py`	WSGI application entry point
`asgi.py`	ASGI application entry point
`__init__.py`	Directory-কে Python package হিসেবে চিহ্নিত করে

7. What Is pip?

`pip` হলো Python package installer।

Common Commands

Install Django:

```
python -m pip install Django
```

List installed packages:

```
python -m pip list
```

Show package details:

```
python -m pip show Django
```

Export installed package versions:

```
python -m pip freeze
```

Save dependencies:

```
python -m pip freeze > requirements.txt
```

Install dependencies from a file:

```
python -m pip install -r requirements.txt
```

Best Practice: `requirements.txt` -এ project-এর প্রয়োজনীয় dependencies রাখবে। বড় project-এ dependency management-এর জন্য `pyproject.toml` বা অন্য উপযুক্ত dependency-management tool-ও ব্যবহার করা যায়।

8. Create a Django App

একটি Django project-এর মধ্যে একাধিক app থাকতে পারে।

Example:

```
myproject/
├── manage.py
├── config/
├── users/
├── products/
├── orders/
└── payments/
```

প্রতিটি app একটি নির্দিষ্ট functionality পরিচালনা করতে পারে।

Create an app:

```
python manage.py startapp products
```

Typical app structure:

```
products/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

Important App Files

File	Purpose
`models.py`	Database models
`views.py`	Request handling and application logic

`admin.py`	Django admin registration
`apps.py`	App configuration
`tests.py`	Automated tests
`migrations/`	Database schema migration files

URL configuration সাধারণত app-এর মধ্যে `urls.py` নামে আলাদা file তৈরি করে রাখা হয়।

9. Register an App

`config/settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "products",
]
```

`INSTALLED_APPS`-এ app register করলে Django সেটিকে project configuration-এর অংশ হিসেবে চিনতে পারে।

10. Django Database Support

Django officially supports several relational database backends, including:

- SQLite
- PostgreSQL
- MySQL
- MariaDB
- Oracle

Supported versions এবং backend-specific requirements-এর জন্য official documentation দেখবে।

SQLite

SQLite একটি lightweight relational database, যা সাধারণত একটি file-এ data সংরক্ষণ করে।

Default database file:

```
db.sqlite3
```

Advantages:

- আলাদা database server প্রয়োজন নেই।
- Local development-এর জন্য সহজ।
- Beginner-friendly।
- Basic testing-এর জন্য সুবিধাজনক।

PostgreSQL

PostgreSQL একটি powerful relational database, যা অনেক production application-এ ব্যবহৃত হয়।

Advantages:

- Advanced SQL features
- Transaction support
- Indexing and query optimization
- Constraints and relationships
- JSON data support
- Concurrent application workloads-এর জন্য উপযুক্ত capabilities

Recommendation: শেখার শুরুতে SQLite ব্যবহার করতে পারো। Production application-এর requirements অনুযায়ী PostgreSQL বা অন্য উপযুক্ত database নির্বাচন করবে।

11. Database Configuration

Django-এর database configuration `settings.py`-তে থাকে।

Default SQLite Configuration

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Important Database Settings

Setting	Purpose
`ENGINE`	Database backend নির্ধারণ করে
`NAME`	Database name অথবা SQLite file path
`USER`	Database username
`PASSWORD`	Database password

`HOST`	Database server address
`PORT`	Database server port

PostgreSQL Configuration Example

প্রথমে driver install করো:

```
python -m pip install "psycopg[binary]"
```

`settings.py`:

```
import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydatabase",
        "USER": "myuser",
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

এই example-এর জন্য `DB_PASSWORD` environment variable configure করতে হবে এবং PostgreSQL database ও user আগে থেকে তৈরি থাকতে হবে।

Security Best Practices:

- Password source code-এ hardcode করবে না।
- Production secrets Git repository-তে commit করবে না।
- Environment variables অথবা secret-management systems ব্যবহার করবে।
- Database user-কে প্রয়োজনের অতিরিক্ত privileges দেবে না।

12. What Is a Migration?

Migration হলো Django model-এর পরিবর্তন database schema-তে প্রয়োগ করার ব্যবস্থা।

Migration Workflow

```
models.py
    |
    v
makemigrations
    |
    v
Migration Files
    |
    v
migrate
    |
```

```
    v
Database Schema
```

`makemigrations`

Models-এর পরিবর্তন বিশ্লেষণ করে migration files তৈরি করে।

```
python manage.py makemigrations
```

Specific app-এর জন্য:

```
python manage.py makemigrations products
```

`migrate`

Migration files database-এ প্রয়োগ করে।

```
python manage.py migrate
```

Other Useful Commands

Show migrations:

```
python manage.py showmigrations
```

Inspect SQL for a migration:

```
python manage.py sqlmigrate products 0001
```

Interview Answer:

- `makemigrations` creates migration files.
- `migrate` applies migrations to the database.

13. Run the Development Server

```
python manage.py runserver
```

Default address:
http://127.0.0.1:8000/
Custom port:

```
python manage.py runserver 8080
```

Important Note

Django development server local development-এর জন্য তৈরি। Production deployment-এ এটি ব্যবহার করা উচিত নয়। Production-এ appropriate WSGI/ASGI server, HTTPS, static file serving এবং secure settings configure করতে হবে।

14. WSGI vs ASGI

WSGI: Web Server Gateway Interface।
ASGI: Asynchronous Server Gateway Interface।

Feature	WSGI	ASGI

Interface	Traditional synchronous interface	Synchronous ও asynchronous handling support করে
Async capabilities	Standard interface-এ সীমিত	Asynchronous application support
WebSockets	Standard WSGI support করে না	Compatible ASGI stack দিয়ে support করা যায়
Example servers	Gunicorn, uWSGI	Uvicorn, Daphne, Hypercorn

WSGI Entry Point

`config/wsgi.py`:

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)

application = get_wsgi_application()
```

ASGI Entry Point

`config/asgi.py`:

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings",
)
```

```
application = get_asgi_application()
```

Important: ASGI ব্যবহার করলেই প্রতিটি view স্বয়ংক্রিয়ভাবে asynchronous হয়ে যায় না। Async behavior-এর জন্য application code, middleware এবং database access patterns-ও উপযুক্ত হতে হবে।

15. Is Apache Required for Django?

না। Local development-এর জন্য Apache অথবা Nginx প্রয়োজন নেই।

Production architecture-এর একটি উদাহরণ:

```
Client / Browser
       |
       v
Reverse Proxy
(Nginx / Apache)
       |
       v
Application Server
(Gunicorn / ASGI Server)
       |
       v
Django Application
       |
       v
Database
```

Static files, media files, caching, TLS termination এবং background jobs-এর architecture project অনুযায়ী ভিন্ন হতে পারে।

16. Complete Django Installation Commands

```
mkdir myproject
cd myproject

python -m venv .venv
```

Windows CMD:

```
.venv\Scripts\activate
```

Linux/macOS:

```
source .venv/bin/activate
```

Install Django:

```
python -m pip install Django
python -m django --version
```

Create project:

```
django-admin startproject config .
```

Apply migrations:

```
python manage.py migrate
```

Start development server:

```
python manage.py runserver
```

Open:
http://127.0.0.1:8000/

Part 2: Django Request Lifecycle

1. What Is the Django Request Lifecycle?

Django Request Lifecycle হলো browser বা অন্য client থেকে আসা HTTP request কীভাবে Django application-এর মধ্য দিয়ে process হয় এবং HTTP response হিসেবে client-এর কাছে ফিরে যায়, সেই সম্পূর্ণ প্রক্রিয়া।

2. Complete Request Lifecycle

```
Browser / API Client
        |
        v
Web Server / ASGI or WSGI Server
        |
        v
Django Handler
        |
        v
Request Middleware
        |
        v
URL Resolver
        |
        v
View
        |
        +----------------------+
        |                      |
        v                      v
      Model                Service
        |                      |
        v                      v
     Database             External API
        |
        v
Template Rendering (if needed)
        |
        v
HttpResponse / JsonResponse
        |
        v
Response Middleware
        |
        v
WSGI / ASGI Server
        |
        v
Web Server / Proxy
        |
```

```
          v
  Browser / API Client
```

Important: এই diagram একটি conceptual overview। প্রতিটি request-এ database query, service call অথবা template rendering বাধ্যতামূলক নয়। URL resolution এবং exception handling-এর মতো কিছু ধাপও পরিস্থিতি অনুযায়ী ভিন্নভাবে কাজ করতে পারে।

3. HTTP Request

HTTP request হলো client-এর পক্ষ থেকে server-কে পাঠানো message।

একটি HTTP request-এ থাকতে পারে:

- HTTP method
- URL path
- Headers
- Cookies
- Query parameters
- Request body

Example:

```
GET /products/5/ HTTP/1.1
Host: example.com
Accept: text/html
```

Common HTTP Methods

Method	Purpose
`GET`	Data retrieve করা
`POST`	Data submit করা অথবা resource তৈরি করা
`PUT`	Resource-এর সম্পূর্ণ representation replace করা
`PATCH`	Resource-এর নির্দিষ্ট অংশ update করা
`DELETE`	Resource delete করা

Django Request Object

Django view-তে প্রথম argument সাধারণত `request`।

```
def my_view(request):
    print(request.method)
    print(request.path)
    print(request.GET)
    print(request.POST)
    print(request.headers)

    ...
```

`request.GET` query parameters ধারণ করে।

`request.POST` সাধারণত form-encoded POST data-এর জন্য ব্যবহৃত হয়। JSON request body পড়তে `request.body` অথবা Django REST Framework-এর `request.data` ব্যবহার করা যায়, context অনুযায়ী।

4. Django Middleware

Middleware হলো request এবং response processing-এর একটি reusable layer।

Middleware দিয়ে:

- Security checks করা যায়।
- Session information পরিচালনা করা যায়।
- Authentication information যুক্ত করা যায়।
- Request reject করা যায়।
- Response headers পরিবর্তন করা যায়।
- Request এবং response logging করা যায়।

Common Middleware

- `SecurityMiddleware`
- `SessionMiddleware`
- `CommonMiddleware`
- `CsrfViewMiddleware`
- `AuthenticationMiddleware`
- `MessageMiddleware`

Middleware Flow

```
Incoming Request
       |
       v
Middleware A
       |
       v
Middleware B
       |
       v
Middleware C
       |
       v
View
       |
       v
Response
```

```
        |
        v
Middleware C
        |
        v
Middleware B
        |
        v
Middleware A
        |
        v
Client
```

এটি সাধারণ flow-এর conceptual representation। কোনো middleware request phase-এ response return করলে পরবর্তী middleware এবং view execute নাও হতে পারে।

5. URL Resolver

Django URL Resolver requested URL-এর সঙ্গে `urls.py` -এর configured patterns match করে উপযুক্ত view নির্বাচন করে।

Example: `urls.py`

```
from django.urls import path

from .views import product_detail

urlpatterns = [
    path(
        "products/<int:id>/",
        product_detail,
        name="product_detail",
    ),
]
```

Request:

```
/products/5/
```

এখানে:

- `products/` হলো static URL segment।
- `<int:id>` হলো integer path converter।
- `5` হলো `id` parameter-এর value।
- `product_detail` হলো selected view।

What If the URL Does Not Match?

সাধারণত Django `404 Not Found` response তৈরি করে।

Interview Answer: URL Resolver matches a requested URL against configured URL patterns and identifies the corresponding view and captured arguments.

6. Function-Based Views (FBV)

Function-Based View হলো Python function দিয়ে request handling করার পদ্ধতি।

Example:

```
from django.http import HttpResponse
```

```
def hello(request):
    return HttpResponse("Hello, World!")
```

URL configuration:

```
from django.urls import path

from .views import hello

urlpatterns = [
    path("hello/", hello, name="hello"),
]
```

7. Database Query in a View

`products/models.py`:

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    def __str__(self):
        return self.name
```

View:

```
from django.shortcuts import get_object_or_404, render

from .models import Product


def product_detail(request, id):
    product = get_object_or_404(Product, pk=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

`get_object_or_404()` object না পেলে `Http404` exception raise করে, যা Django সাধারণত `404 Not Found` response-এ রূপান্তর করে।

8. Class-Based Views (CBV)

Class-Based View হলো class ব্যবহার করে view behavior organize করার পদ্ধতি।

Example:

```
from django.views.generic import DetailView
```

```
from .models import Product


class ProductDetailView(DetailView):
    model = Product
    template_name = "product.html"
```

URL configuration:

```
from django.urls import path

from .views import ProductDetailView

urlpatterns = [
    path(
        "products/<int:pk>/",
        ProductDetailView.as_view(),
        name="product_detail",
    ),
]
```

FBV vs CBV

Feature	FBV	CBV
Structure	Function	Class
Learning curve	সাধারণত সহজ	Inheritance ও dispatch বোঝা প্রয়োজন
Reuse	Helper functions ও decorators	Inheritance ও mixins
Best suited for	Simple বা custom logic	Reusable generic patterns ও structured views

দুটিই valid। কোনটি ব্যবহার করবে তা project requirements এবং maintainability-এর ওপর নির্ভর করে।

9. Django ORM

ORM-এর পূর্ণরূপ **Object-Relational Mapper**।

Django ORM দিয়ে Python code ব্যবহার করে relational database-এর সঙ্গে কাজ করা যায়।

Retrieve One Object

```
product = Product.objects.get(pk=5)

print(product.name)
print(product.price)
```

`get()` -এর গুরুত্বপূর্ণ behavior:

- কোনো object না পেলে `Product.DoesNotExist` exception।
- একাধিক object match করলে `Product.MultipleObjectsReturned` exception।

Retrieve Multiple Objects

```
products = Product.objects.all()
```

Filter objects:

```
products = Product.objects.filter(price__gte=100)
```

Create an object:

```
product = Product.objects.create(
    name="Keyboard",
    price="1500.00",
)
```

Update an object:

```
product = Product.objects.get(pk=5)
product.price = "1800.00"
product.save()
```

Delete an object:

```
product = Product.objects.get(pk=5)
product.delete()
```

Important ORM Concepts

- `all()`
- `filter()`
- `exclude()`
- `get()`
- `create()`
- `update()`
- `delete()`
- `order_by()`
- `values()`
- `select_related()`
- `prefetch_related()`

Best Practice: Query optimization-এর জন্য প্রয়োজন অনুযায়ী `select_related()` এবং `prefetch_related()` ব্যবহার করবে। তবে query count ও performance যাচাই করে optimization করবে।

10. Template Rendering

Django Template Engine দিয়ে dynamic HTML তৈরি করা যায়।

`product.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Product Details</title>
</head>
<body>
    <h1>{{ product.name }}</h1>
    <p>Price: {{ product.price }}</p>
</body>
</html>
```

View

```
from django.shortcuts import get_object_or_404, render

from .models import Product


def product_detail(request, id):
    product = get_object_or_404(Product, pk=id)

    return render(
        request,
        "product.html",
        {"product": product},
    )
```

`render()` template এবং context data ব্যবহার করে rendered content তৈরি করে এবং `HttpResponse` return করে।

Important Template Syntax

Variable:

```
{{ product.name }}
```

Conditional:

```
{% if product %}
    <h1>{{ product.name }}</h1>
{% else %}
    <p>Product not found.</p>
{% endif %}
```

Loop:

```
{% for product in products %}
    <p>{{ product.name }}</p>
{% endfor %}
```

11. HTTP Response Types

Response	Purpose

`HttpResponse`	সাধারণ text বা HTML response
`JsonResponse`	JSON response
`HttpResponseRedirect`	Redirect response
`StreamingHttpResponse`	Streaming response
`TemplateResponse`	Template-based response

HttpResponse

```
from django.http import HttpResponse


def hello(request):
    return HttpResponse("Hello, World!")
```

JsonResponse

```
from django.http import JsonResponse


def api(request):
    return JsonResponse({
        "name": "Faruk",
        "role": "Backend Developer",
    })
```

Redirect

```
from django.shortcuts import redirect


def my_view(request):
    return redirect("home")
```

এখানে `home` হলো URL pattern-এর name।

Redirect response সাধারণত `Location` header দিয়ে browser-কে পরবর্তী URL জানায়।

render() vs JsonResponse()

Feature	`render()`	`JsonResponse()`
Output	Rendered HTML	JSON
Typical use	Server-rendered web pages	Simple JSON endpoints
Input	Template and context	JSON-compatible data

Django REST Framework ব্যবহার করলে API response-এর জন্য `rest_framework.response.Response`-ও ব্যবহার করা যায়।

12. Middleware Order

Middleware-এর order গুরুত্বপূর্ণ, কারণ একটি middleware অন্য middleware-এর তৈরি করা information-এর ওপর নির্ভর করতে পারে।

Example:

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

`AuthenticationMiddleware` session information ব্যবহার করে authenticated user-এর তথ্য `request.user`-এ available করে।

সাধারণভাবে request middleware chain-এর মধ্য দিয়ে সামনে যায় এবং response reverse order-এ ফিরে আসে। Exception handling ও short-circuit behavior flow-কে প্রভাবিত করতে পারে।

13. Django Request Lifecycle Interview Questions

Q1. What Is the Django Request Lifecycle?

Browser বা API client থেকে HTTP request আসার পর Django request processing করে এবং HTTP response client-এ ফেরত পাঠানোর সম্পূর্ণ প্রক্রিয়া।

Q2. What Does the URL Resolver Do?

Configured URL patterns match করে উপযুক্ত view এবং captured arguments নির্ধারণ করে।

Q3. What Is Middleware?

Request এবং response processing-এর reusable component।

Q4. Can Middleware Block a Request?

হ্যাঁ। Middleware view execute হওয়ার আগেই response return করতে পারে।

Q5. What Is Django ORM?

Python objects এবং QuerySets ব্যবহার করে database interaction করার interface।

Q6. What Does `render()` Do?

Template এবং context data ব্যবহার করে rendered HTTP response তৈরি করে।

Q7. What Is the Difference Between WSGI and ASGI?

WSGI একটি traditional synchronous interface। ASGI synchronous ও asynchronous application handling এবং compatible stack-এর মাধ্যমে WebSockets-এর মতো protocol support করতে পারে।

Q8. Why Does Middleware Order Matter?

Middleware dependencies এবং request/response processing-এর সঠিক ক্রম বজায় রাখার জন্য।

Q9. Does Every Request Need a Database Query?

না। Static response বা অন্য ধরনের response database access ছাড়াও তৈরি করা যায়।

Q10. Does Every Request Need Template Rendering?

না। API view সরাসরি JSON response দিতে পারে।

Q11. What Happens If a URL Does Not Match?

সাধারণত Django `404 Not Found` response তৈরি করে।

Q12. What Is the Difference Between FBV and CBV?

FBV function ব্যবহার করে; CBV class এবং class-based dispatch ব্যবহার করে।

Part 3: Reusable Django Apps and Python Packaging

1. What Is a Reusable Django App?

Reusable Django app হলো এমন একটি Django application, যা একাধিক Django project-এ পুনরায় ব্যবহার করা যায়।

Examples

- Authentication utilities
- Custom permissions
- API response helpers
- Pagination
- Logging
- Notifications
- Audit trails
- Common validators

Advantages

- Code reusability
- Less duplication
- Easier maintenance
- Better testing
- Consistent application behavior
- Faster development

Important: সব utility reusable Django app হওয়া বাধ্যতামূলক নয়। কিছু utility সাধারণ Python package হিসেবেও তৈরি করা যায়।

2. Create a Reusable Django App

Step 1: Create the Project

```
mkdir django-reusable-learning
cd django-reusable-learning

python -m venv .venv
```

Activate the environment using the appropriate command for your operating system.

Step 2: Install Dependencies

```
python -m pip install Django djangorestframework
```

Step 3: Create a Project and App

```
django-admin startproject config .
python manage.py startapp common_api
```

Step 4: Register the App

`config/settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "rest_framework",
    "common_api",
]
```

3. Create Reusable API Response Helpers

Create `common_api/responses.py`:

```
from rest_framework.response import Response


def success_response(
    data=None,
    message="Success",
    status_code=200,
):
    return Response(
        {
            "success": True,
            "message": message,
            "data": data,
        },
        status=status_code,
    )


def error_response(
    message="Something went wrong",
    status_code=400,
```

```
    data=None,
):
    return Response(
        {
            "success": False,
            "message": message,
            "data": data,
        },
        status=status_code,
    )
```

Explanation

- `Response` : Django REST Framework-এর response class।
- `data` : Response-এর payload।
- `message` : Human-readable message।
- `status_code` : HTTP status code।
- `success` : Application-level result indicator।

Technical Note: এই functions সাধারণ DRF `Response` objects তৈরি করে। এগুলো automatic validation, exception handling অথবা authentication implement করে না।

4. Use the Response Helpers in API Views

Create `common_api/views.py` :

```
from rest_framework.views import APIView

from .responses import error_response, success_response


class DemoAPIView(APIView):
    def get(self, request):
        user = {
            "id": 1,
            "name": "Faruk",
            "role": "Backend Developer",
        }

        return success_response(
            data=user,
            message="User retrieved successfully",
        )


class ErrorDemoAPIView(APIView):
    def get(self, request):
        return error_response(
            message="User not found",
            status_code=404,
        )
```

5. Configure API URLs

`config/urls.py` :

```
from django.contrib import admin
from django.urls import path

from common_api.views import DemoAPIView, ErrorDemoAPIView


urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/demo/", DemoAPIView.as_view(), name="demo-api"),
    path("api/error/", ErrorDemoAPIView.as_view(), name="error-api"),
]
```

Start the development server:

```
python manage.py runserver
```

Test endpoints:

- http://127.0.0.1:8000/api/demo/
- http://127.0.0.1:8000/api/error/

Success Response Example

```
{
    "success": true,
    "message": "User retrieved successfully",
    "data": {
        "id": 1,
        "name": "Faruk",
        "role": "Backend Developer"
    }
}
```

Error Response Example

```
{
    "success": false,
    "message": "User not found",
    "data": null
}
```

The second endpoint returns HTTP status `404`.

Best Practice: API response structure consistent রাখবে। তবে প্রতিটি API-তে custom success/error wrapper ব্যবহার করতেই হবে এমন নয়। Project conventions এবং client requirements অনুযায়ী design করবে।

6. Test the API

Create or update `common_api/tests.py`:

```
from rest_framework.test import APITestCase


class DemoAPITest(APITestCase):
    def test_demo_api(self):
        response = self.client.get("/api/demo/")

        self.assertEqual(response.status_code, 200)
```

```
        self.assertTrue(response.data["success"])
        self.assertEqual(
            response.data["data"]["name"],
            "Faruk",
        )

    def test_error_api(self):
        response = self.client.get("/api/error/")

        self.assertEqual(response.status_code, 404)
        self.assertFalse(response.data["success"])
```

Run tests:

```
python manage.py test common_api
```

Why Do We Write Tests?

- Expected behavior verify করার জন্য।
- Bugs দ্রুত শনাক্ত করার জন্য।
- Refactoring-এর সময় confidence বজায় রাখার জন্য।
- Regression risk কমানোর জন্য।
- Team collaboration উন্নত করার জন্য।

Important: Test endpoint, response data এবং status codes যাচাই করছে। Production API-তে authentication, permissions, validation এবং failure cases-এর জন্যও tests লিখবে।

7. What Is Python Packaging?

Python packaging হলো Python code organize, build এবং distribute করার প্রক্রিয়া, যাতে অন্য project সহজে install ও ব্যবহার করতে পারে।

Important Terms

Term	Meaning
Module	Python file, যেমন `responses.py`
Package	Importable Python modules-এর collection
`pip`	Python package installer
PyPI	Python Package Index

`pyproject.toml`	Project metadata এবং build configuration
Wheel	Built distribution archive, সাধারণত `.whl`
Source distribution	Source code distribution archive
Editable install	Development-এর জন্য source-linked installation

Reusable Django App vs Python Package

Reusable Django App: Reusable Django functionality এবং প্রয়োজনে models, migrations, views, templates ও app configuration ধারণ করতে পারে।

Python Package: Python code installable distribution হিসেবে organize ও distribute করার পদ্ধতি।

একটি reusable Django app-কে Python package হিসেবে distribute করা যায়।

8. Create a Python Package

Recommended structure:

```
django-common-toolkit/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── common_api/
│       ├── __init__.py
│       ├── apps.py
│       └── responses.py
└── tests/
```

Distribution Name vs Import Name

Example:

- Distribution name: `django-common-toolkit-faruk`
- Import name: `common_api`

Distribution name `pip install`-এ ব্যবহৃত হয়, আর import name Python code-এ ব্যবহৃত হয়।

`apps.py`

```
from django.apps import AppConfig


class CommonApiConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "common_api"
```

এই configuration-টি `common_api` import package-এর সঙ্গে সম্পর্কিত। Django app-এর package layout এবং distribution structure সঠিকভাবে configure করতে হবে।

9. Configure `pyproject.toml`

Package root-এ `pyproject.toml` তৈরি করো:

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "django-common-toolkit-faruk"
version = "0.1.0"
description = "Reusable API utilities for Django"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "Django>=4.2",
    "djangorestframework>=3.14",
]

[tool.setuptools.packages.find]
where = ["src"]
```

Important Fields

Field	Purpose
`build-system`	Build backend এবং build requirements
`name`	Distribution package name
`version`	Package version

`description`	Package-এর সংক্ষিপ্ত পরিচিতি
`readme`	Documentation file
`requires-python`	Minimum supported Python version
`dependencies`	Required runtime dependencies
`where`	Package discovery-এর source directory

Compatibility Note

উপরের dependency versions example হিসেবে দেওয়া হয়েছে। Release করার আগে Django এবং Django REST Framework-এর supported versions, Python compatibility এবং project-এর tested configuration যাচাই করবে।

10. Install the Package Locally

Package root থেকে:

```
python -m pip install -e .
```

`-e` অর্থ editable installation।

Development-এর সময় source code পরিবর্তন করলে প্রতিবার package rebuild ও reinstall করার প্রয়োজন সাধারণত হয় না।

Import the Package

```
from common_api.responses import success_response
```

যদি package একটি Django app হয়, প্রয়োজন অনুযায়ী `INSTALLED_APPS`-এ register করবে।

Important `src/` Layout Note

উপরের package structure-এর ক্ষেত্রে package discovery `src/` directory থেকে করা হচ্ছে। Editable install-এর মাধ্যমে development environment-এ package import করা যায়।

Wheel এবং source distribution-এর মধ্যে package files সঠিকভাবে অন্তর্ভুক্ত হয়েছে কি না যাচাই করা গুরুত্বপূর্ণ।

11. Build the Package

Install build tools:

```
python -m pip install build
```

Build distributions:

```
python -m build
```

Generated files সাধারণত `dist/` directory-তে থাকবে:

```
dist/
├── django_common_toolkit_faruk-0.1.0-py3-none-any.whl
└── django_common_toolkit_faruk-0.1.0.tar.gz
```

Distribution Types

Wheel (`.whl`): Built distribution format, যা সাধারণত install করা সহজ করে।

Source Distribution (`.tar.gz`): Source files এবং build-related metadata ধারণ করে, যা থেকে distribution তৈরি করা যায়।

12. Validate the Package

Install Twine:

```
python -m pip install twine
```

Validate distributions:

```
python -m twine check dist/*
```

Recommended Validation Steps

1. Automated tests চালাও।
2. Package build করো।
3. Distribution metadata যাচাই করো।
4. Wheel ও source distribution contents পরীক্ষা করো।
5. একটি clean virtual environment তৈরি করো।
6. সেই environment-এ built package install করো।
7. Import এবং functionality test করো।
8. Django integration প্রয়োজন হলে একটি fresh Django project-এ পরীক্ষা করো।

Important: Editable install সফল হওয়া মানেই built distribution-ও সঠিকভাবে কাজ করবে—এমন নিশ্চয়তা নেই।

13. Publish to PyPI

Recommended publishing workflow:

1. PyPI account তৈরি করো।
2. README, license এবং metadata প্রস্তুত করো।
3. Automated tests চালাও।
4. Package build করো।
5. Distribution validate করো।
6. TestPyPI-তে publishing ও installation পরীক্ষা করো।
7. PyPI-এর supported authentication method ব্যবহার করো।
8. Release install করে fresh environment-এ পরীক্ষা করো।
9. Version tag এবং release notes তৈরি করো।

Official resources:

- Python Packaging Tutorial: https://packaging.python.org/en/latest/tutorials/packaging-projects/
- PyPI: https://pypi.org/
- TestPyPI: https://test.pypi.org/

Security

- API tokens source control-এ commit করবে না।

- Publishing credentials environment বা secure secret store-এ রাখবে।
- Release-এর আগে package contents যাচাই করবে।
- Dependency versions এবং license requirements review করবে।

14. Semantic Versioning

Semantic Versioning-এর সাধারণ format:

```
MAJOR.MINOR.PATCH
```

Version Component	Meaning	Example
MAJOR	Breaking changes	`1.0.0` → `2.0.0`
MINOR	Backward-compatible features	`1.0.0` → `1.1.0`
PATCH	Backward-compatible bug fixes	`1.0.0` → `1.0.1`

Note: Semantic Versioning অনুসরণ করলে consumers বুঝতে পারে release-এর পরিবর্তনের ধরন কী হতে পারে।

15. Best Practices for Reusable Django Apps

- DRY principle অনুসরণ করো।
- প্রতিটি app-কে একটি নির্দিষ্ট responsibility দাও।
- Project-specific settings hardcode করবে না।
- Automated tests লিখবে।
- README এবং installation instructions রাখবে।
- Dependency compatibility পরীক্ষা করবে।
- Git এবং release tags ব্যবহার করবে।
- Models থাকলে migrations অন্তর্ভুক্ত করবে।
- Secrets source code-এ রাখবে না।
- Package অন্য project-এ পরীক্ষা করবে।
- Public APIs পরিবর্তনের সময় backward compatibility বিবেচনা করবে।
- প্রয়োজন না হলে unnecessary dependencies যোগ করবে না।

Final Revision Checklist

Django Learning Progress

- Virtual environment তৈরি ও activate করতে পারি।

- Django install এবং version check করতে পারি।
- Django project ও app তৈরি করতে পারি।
- `settings.py` এবং `INSTALLED_APPS` বুঝি।
- Database configuration করতে পারি।
- `makemigrations` এবং `migrate`-এর পার্থক্য জানি।
- WSGI এবং ASGI-এর পার্থক্য ব্যাখ্যা করতে পারি।
- Django request lifecycle ব্যাখ্যা করতে পারি।
- Middleware এবং URL Resolver-এর কাজ বুঝি।
- FBV এবং CBV-এর basic syntax লিখতে পারি।
- Django ORM দিয়ে CRUD করতে পারি।
- `HttpResponse`, `JsonResponse` এবং `render()` বুঝি।
- Reusable API response functions তৈরি করতে পারি।
- Django REST Framework API-এর automated tests লিখতে পারি।
- `pyproject.toml` দিয়ে Python package configure করতে পারি।
- Package build, validate এবং locally install করতে পারি।
- একটি clean environment-এ package test করতে পারি।

Recommended Learning Roadmap

Backend Developer হিসেবে Django শেখার জন্য নিচের ক্রম অনুসরণ করতে পারো:

1. **Django Fundamentals** — Projects, apps, settings, URLs, views, templates।
2. **Models and ORM** — CRUD, relationships, QuerySets, migrations।
3. **Forms and Authentication** — Validation, login, permissions, sessions।
4. **Django REST Framework** — Serializers, APIViews, ViewSets, routers।
5. **Authentication and Authorization** — JWT, permissions, role-based access।
6. **PostgreSQL** — Schema design, indexes, constraints, transactions।
7. **Query Optimization** — `select_related()`, `prefetch_related()`, query analysis।
8. **Automated Testing** — Django TestCase, DRF APITestCase, pytest।
9. **Reusable Apps and Packaging** — `pyproject.toml`, builds, releases।
10. **Deployment** — Production settings, Gunicorn/Uvicorn, HTTPS, logging।
11. **Advanced Backend Topics** — Caching, background tasks, security, monitoring।

Recommended Practice Project: Task Management REST API

একটি real-world Task Management API তৈরি করো, যেখানে থাকবে:

- User registration and login
- JWT authentication
- Task CRUD operations
- Serializer validation
- Pagination and filtering
- User-specific permissions
- PostgreSQL integration
- Automated API tests
- Environment-based configuration
- API documentation
- Production deployment

Final Advice: শুধু notes মুখস্থ না করে প্রতিটি concept নিজে code করে practice করবে। Interview-এ concept explain করার পাশাপাশি বাস্তব code, debugging এবং design decisions বোঝাতে পারলে বেশি উপকার হবে।

Official Documentation

- Django Documentation: https://docs.djangoproject.com/en/
- Django Tutorial: https://docs.djangoproject.com/en/stable/intro/tutorial01/
- Django REST Framework: https://www.django-rest-framework.org/
- Python Virtual Environments: https://docs.python.org/3/library/venv.html
- Python Packaging Guide: https://packaging.python.org/en/latest/
- PyPI: https://pypi.org/